

UNIVERSIDADE FEDERAL DO RIO DE JANEIRO
ESCOLA POLITÉCNICA
DEPARTAMENTO DE ENGENHARIA ELETRÔNICA E DE
COMPUTAÇÃO

PROPOSTA DE PROJETO DE GRADUAÇÃO

Aluno: Matheus Hoffmann Fernandes Santos
hoffmann@poli.ufrj.br

Orientador: A definir, Dr.

Coorientador: Pedro Veras Guimarães, Dr. (LabECO, Departamento de
Engenharia Mecânica, UFSC)

1. TÍTULO

Otimização operacional do modelo de ondas WAVEWATCH III® no ciclo de previsão do ReNOMO (LabECO/UFSC): opções de compilação, configuração, Fortran moderno e viabilidade em GPU.

2. ÊNFASE

Computação / Computação científica de alto desempenho.

3. TEMA

Modelos de ondas de terceira geração resolvem a equação de balanço da ação das ondas sobre uma grade espacial e uma grade espectral, integrando em cada ponto e passo de tempo a entrada de vento, as interações não lineares e a dissipação. O WAVEWATCH III® (WW3), desenvolvido pelo NOAA/NCEP, é o modelo de referência dessa classe e o adotado pela maioria dos centros operacionais (The WAVEWATCH III Development Group, 2019). É escrito em Fortran com MPI e OpenMP, e um arquivo de *switches* fixa em tempo de compilação a física, o esquema de propagação e as opções de entrada e saída.

O Laboratório de Engenharia e Ciências Oceânicas da Universidade Federal de Santa Catarina (LabECO/UFSC) executa o WW3 como parte de sua contribuição à Rede Nacional de Observação e Monitoramento Oceânico (ReNOMO), financiada pela chamada CNPq/MCTI/Finep 062/2022, que integra observação, controle de qualidade e produtos operacionais para o Atlântico Sul e a costa brasileira (LabECO/UFSC, 2026). Em um ciclo operacional, o custo de uma previsão é o seu

tempo de execução (*wall-clock*), que determina quantas vezes por dia o modelo pode rodar, em que resolução e com quantos domínios aninhados ou membros de conjunto.

O tema deste projeto é a redução sistemática desse tempo de execução na configuração operada pelo LabECO, tratada como um problema de engenharia com resultados mensuráveis: um *benchmark* reproduível da execução atual, um perfil que mostre onde o tempo é gasto e uma sequência de otimizações de profundidade crescente, das opções de compilação à refatoração dirigida das rotinas mais custosas em Fortran moderno, complementada por um estudo de viabilidade em GPU (NVIDIA H100) dimensionado pelo cronograma do WAVEWATCH IV. Cada etapa é medida na configuração do próprio laboratório e só é aceita dentro de tolerâncias acordadas com os pesquisadores: a cadeia operacional nunca troca correção por velocidade.

4. DELIMITAÇÃO

O trabalho restringe-se ao *driver* de grade única do WW3 (`ww3_shel`) na configuração operada pelo LabECO: sua grade, seu arquivo de *switches*, sua forçante e sua lista de saídas. Essa configuração e a revisão exata do código-fonte são congeladas e documentadas no início do projeto, de modo que todas as medidas se refiram ao mesmo caso.

Quatro etapas estão no escopo, executadas em ordem e cada uma validada por comparação com a execução de referência:

1. **Opções de compilação.** Compilador, *flags* de otimização, arquivo de *switches* (em particular memória compartilhada, MPI ou híbrido, e a saída NetCDF-4) e a distribuição de processos MPI e *threads* OpenMP. Não altera o código-fonte; espera-se reprodução bit a bit da referência ou, com reordenação de operações em ponto flutuante, diferenças da ordem do arredondamento.
2. **Configuração da execução.** Decomposição do domínio, passos de tempo permitidos pela condição CFL, frequência e campos de saída, estratégia de *restart* e interpolação da forçante. Altera apenas *namelists*.
3. **Refatoração dirigida em Fortran moderno.** Reescrita das rotinas que o perfil identificar como dominantes, tipicamente a integração dos termos de fonte (W3SRCE) e a propagação espacial, com construções padrão Fortran

2008/2018 (do `concurrent`, interfaces explícitas, vetores contíguos), sem alterar o algoritmo numérico e dentro de uma tolerância definida por campo de saída.

4. **Estudo de viabilidade em GPU (NVIDIA H100).** Um estudo, não um *port*: quais *kernels* podem executar no dispositivo, que dados precisariam residir nele e qual ganho é plausível, com base nos *ports* publicados do WW3 e do WAM e em um protótipo de um único *kernel* sobre a camada de portabilidade Kokkos (Trott et al., 2022). O resultado é uma recomendação com medidas, não um *build* operacional.

A etapa 4 é deliberadamente a menor: o WAVEWATCH IV (WW4), reescrita do modelo com arquitetura voltada a GPU desde a origem, tem primeiro lançamento previsto para janeiro de 2027, e um *port* completo do WW3 concorreria com esse trabalho. Ficam fora do escopo o *driver* multigrade (`ww3_multi`), acoplamento com outros modelos, assimilação de dados, grades não estruturadas (PDLIB), salvo se a grade operacional as exigir, alterações nas parametrizações físicas e qualquer contribuição ao WW4 ou dependência dele.

5. JUSTIFICATIVA

Uma cadeia operacional de previsão vale pela sua pontualidade. Para o campo de ondas que alimenta os produtos do ReNOMO, um ciclo mais curto significa entrega mais cedo, espaço para maior resolução no mesmo prazo e a possibilidade de rodar vários membros para quantificar incerteza. O WW3 foi projetado para os supercomputadores das duas últimas décadas e carrega, no sistema de compilação, na organização dos dados e na decomposição paralela, escolhas que não são automaticamente as melhores para as máquinas de um laboratório universitário hoje: nós com muitos núcleos e muita memória, armazenamento NVMe e, cada vez mais, GPUs.

As duas primeiras etapas justificam-se pela documentação do próprio modelo: o arquivo de *switches* e as opções do compilador decidem qual código é compilado e como, e a distribuição paralela decide quanto tempo se gasta movendo espectros entre processos em vez de calculá-los (The WAVEWATCH III Development Group, 2019). São etapas baratas, frequentemente negligenciadas em instalações configuradas uma vez e nunca revisitadas, e produzem o instrumento de que as etapas

seguintes dependem: um *benchmark* reproduzível e um perfil.

A terceira etapa justifica-se pela estrutura do código. Perfis publicados do WW3 mostram que a integração dos termos de fonte domina o tempo de execução, seguida da propagação, e que as rotinas envolvidas usam grandes vetores locais e estado global em nível de módulo, o que dificulta tanto a vetorização quanto qualquer forma de *offload* (Ikuyajolu et al., 2023). Reescrevê-las em Fortran moderno, preservando algoritmo e aritmética, é uma tarefa de engenharia bem compreendida, de valor diretamente mensurável no modelo que o laboratório opera.

A quarta etapa responde à pergunta que o laboratório de fato faz, se um H100 poderia rodar o modelo. O WW3 não tem caminho de execução em GPU na distribuição oficial. O único *port* publicado levou os termos de fonte à GPU com OpenACC e obteve cerca de 1,3 vez de ganho por nó, limitado pelo tráfego CPU–GPU a cada passo de tempo (Ikuyajolu et al., 2023). Já o WAM6-GPU, que refatorou o modelo inteiro para manter os campos residentes no dispositivo, reportou ganho de uma ordem de grandeza em um nó com várias GPUs (Yuan et al., 2024). A lição é que o *offload* parcial não compensa e que um *port* real do WW3 exige reescrever sua organização de dados. Que essa reescrita é possível, e como, deixou de ser conjectura: Koldunov et al. (2026) levaram o FESOM2, com cerca de 74 mil linhas de Fortran, a C e depois a C++/Kokkos em semanas, com um assistente de código baseado em LLM dirigido por especialistas, tradução literal em dois estágios e validação em escada. O protótipo deste projeto segue essa receita, e as três primeiras etapas produzem o que ela exige: configuração congelada, perfil de referência e critério de paridade por rotina.

A relação com o WAVEWATCH IV (WW4) delimita o projeto. O WW4 é a reescrita do modelo anunciada pelo NOAA/NCEP, com núcleo em C++ e arquitetura voltada a CPU e GPU desde a origem; encontra-se em estágio pré-alfa, com primeiro lançamento previsto para janeiro de 2027, e o próprio plano do NOAA só descontinua o suporte ao WW3 quando o WW4 amadurecer (NOAA-EMC, 2026; NOAA/NWS/NCEP, 2025). Para o laboratório, isso significa que o WW3 seguirá operacional por anos, e que qualquer *port* do WW3 para GPU concorre com o que a equipe do WW4 já constrói. Por isso este projeto concentra o esforço na operação do modelo existente e limita a GPU a um estudo, concluído após o lançamento do

WW4, para que a recomendação final compare os dois caminhos em vez de duplicar um deles.

6. OBJETIVO

O objetivo geral é reduzir, de forma medida e reproduzível, o tempo de execução do ciclo operacional do WAVEWATCH III mantido pelo LabECO/UFSC no âmbito do ReNOMO, sem alterar seus resultados científicos além das tolerâncias acordadas com o laboratório, e entregar uma recomendação escrita sobre até onde essa redução pode ser levada com o hardware disponível, incluindo uma GPU NVIDIA H100.

Os objetivos específicos são:

1. Congelar e documentar a configuração operacional (revisão do código, *switches*, *namelists*, grade, forçante, saídas, hardware) e construir um *benchmark* reproduzível da execução de referência, com métrica definida (tempo de execução por hora de previsão).
2. Produzir um perfil da execução de referência por rotina e por fase (termos de fonte, propagação, comunicação, entrada e saída), com um e com vários processos MPI.
3. Quantificar o ganho de cada etapa de otimização, cada uma com sua evidência de paridade em relação à referência, e entregar ao laboratório a melhor configuração como um *build* documentado.
4. Avaliar a viabilidade de execução em GPU em um H100 por meio de um estudo e de um protótipo de *kernel* único, reportando números medidos, as restrições de movimentação de dados e a posição desse resultado frente ao primeiro lançamento do WW4.
5. Publicar ferramentas, resultados e recomendação no repositório aberto, em forma que o laboratório possa reexecutar.

7. METODOLOGIA

O projeto segue o princípio de medir antes de otimizar: nenhuma otimização é tentada antes que a execução de referência seja reproduzível e perfilada, nem aceita sem verificação de paridade contra ela.

Referência e *benchmark*. O caso operacional do laboratório é capturado

como um teste autocontido: arquivos de entrada, forçante para um período fixo, arquivo de *switches*, *namelists* e resultados de referência. O mesmo procedimento é aplicado ao teste de regressão oficial mais próximo da configuração operacional, para comparação com casos da comunidade. O *script* de *benchmark* registra tempo por fase, processos e *threads*, compilador e opções e a revisão do código, e produz uma tabela anexável aos relatórios.

Perfilamento. A execução de referência é perfilada com *gprof* ou *perf* com um, quatro e dezesseis processos MPI, com o tempo de comunicação registrado separadamente. O resultado é uma tabela de tempo por rotina, agrupada nas fases do laço temporal, que define a ordem da refatoração e a base de todas as medidas posteriores.

Crítérios de paridade. Nas etapas 1 e 2, os resultados são comparados bit a bit; quando uma opção do compilador reordena operações em ponto flutuante, a comparação usa tolerâncias por campo (por exemplo, altura significativa, período de pico e direção média), propostas pelo aluno e aprovadas pelo coorientador. Na etapa 3, valem as mesmas tolerâncias e cada rotina reescrita é testada isoladamente contra a original em entradas capturadas. Uma alteração que falha no critério não é incorporada.

Etapas. As opções de compilação são exploradas primeiro, como uma matriz de compilador, opções e distribuição paralela. Segue a configuração da execução, variando um parâmetro de *namelist* por vez. A refatoração ataca as rotinas do topo do perfil, uma de cada vez, em um *fork* que mantém a rotina original acessível por uma chave de tempo de execução no mesmo binário. O estudo de GPU revisa os *ports* publicados, mede o tráfego CPU–GPU por passo de tempo na configuração do laboratório e prototipa um *kernel* (o termo de interação não linear, local por ponto e autocontido) pela receita do FESOM2 (Koldunov et al., 2026): tradução literal para C, verificada contra o Fortran em entradas capturadas; versão Kokkos cujo *backend* serial deve ser idêntico bit a bit ao C; e só então o *backend* CUDA, comparado estatisticamente e cronometrado no H100. Um assistente de código baseado em LLM é usado na tradução sob as mesmas regras, com o aluno e o coorientador responsáveis pela validação, e os *prompts* e a evidência de paridade são versionados com o código. O estudo é concluído após o primeiro lançamento do WW4, previsto

para janeiro de 2027, para que a recomendação compare o que é possível sobre o WW3 com o que o WW4 já entrega em GPU.

Ferramentas e reprodutibilidade. Todo o trabalho é feito no repositório aberto **ww-lab** (Hoffmann, 2026), que fornece a cadeia de ferramentas fixada, os *scripts* de compilação e de teste de regressão, o modelo como submódulo em revisão fixa e integração contínua, de modo que qualquer medida pode ser repetida pelo laboratório em suas próprias máquinas. Os resultados são registrados como tabelas e gráficos gerados por *scripts*, nunca à mão.

Entregas e validação. As entregas são o registro de configuração, as tabelas de *benchmark* e de perfil, o melhor *build* documentado, o memorando de viabilidade em GPU e o relatório final. A validação científica cabe ao coorientador no LabECO, que define as tolerâncias e revisa a evidência de paridade; a de engenharia, pelos testes automatizados do repositório e por reexecuções independentes no laboratório.

Riscos. O principal é a divergência silenciosa de resultados após uma refatoração, mitigada pelos critérios de paridade e por testes unitários por rotina. O segundo é o crescimento do escopo em direção a um *port* completo para GPU, mitigado pela delimitação e pelo cronograma do WW4. O terceiro é a grade operacional ser não estruturada; nesse caso a etapa 3 restringe-se aos termos de fonte.

8. CRONOGRAMA

O projeto ocupa dois períodos letivos; as datas serão ajustadas com os orientadores.

Etapa	Prazo
Revisão bibliográfica; registro da configuração operacional	10/2026
<i>Benchmark</i> reprodutível e perfil da execução de referência	11/2026
Etapa 1: opções de compilação (compilador, <i>flags</i> , <i>switches</i> , paralelismo)	12/2026
Etapa 2: configuração da execução; primeiro lançamento do WW4	01/2027
Etapa 3: refatoração dirigida em Fortran moderno das rotinas dominantes	02/2027
Etapa 4: estudo de viabilidade em GPU e protótipo de <i>kernel</i> no H100	03/2027
Recomendação escrita ao LabECO; relatório final e revisão	04/2027
Defesa (a partir de)	05/2027

Referências Bibliográficas

HOFFMANN, Matheus. **ww-lab: a bootstrap repository for WAVEWATCH III**. <https://github.com/hoffmann/ww-lab>, 2026.

IKUYAJOLU, Olawale James *et al.* **Porting the WAVEWATCH III (v6.07) wave action source terms to GPU**. *Geoscientific Model Development*, v. 16, p. 1445–1458, 2023.

KOLDUNOV, Nikolay V. *et al.* **An Ocean Model Ported by a Large Language Model: Experience and Lessons from FESOM2 (Fortran to C to C++/Kokkos)**. arXiv:2606.11356, 2026.

LABECO/UFSC. **ReNOMO – Rede Nacional de Observação e Monitoramento Oceânico**. <https://labeco.ufsc.br/renomo/>, 2026.

NOAA-EMC. **WAVEWATCH IV (WW4) repository**. <https://github.com/NOAA-EMC/WW4>, 2026.

NOAA/NWS/NCEP. **WAVEWATCH IV: plan for the next-generation wave model**. *[S.l.]*: NCEP, 2025.

THE WAVEWATCH III DEVELOPMENT GROUP. **User manual and system documentation of WAVEWATCH III version 6.07**. College Park, MD, USA: NOAA/NWS/NCEP/MMAB, 2019.

TROTT, Christian R. *et al.* **Kokkos 3: Programming model extensions for the exascale era**. *IEEE Transactions on Parallel and Distributed Systems*, v. 33, n. 4, p. 805–817, 2022.

YUAN, Ye *et al.* **WAM6-GPU v1.0: GPU-accelerated third-generation wave model**. *Geoscientific Model Development*, v. 17, p. 6123–6136, 2024.

Rio de Janeiro, 13 de setembro de 2026.

Matheus Hoffmann Fernandes Santos - Aluno

A definir, Dr. - Orientador

Pedro Veras Guimarães, Dr. - Coorientador